

猫眼影院低票价订阅模块改造总结（供 Agent 直接执行）

一、项目目标

当前 Go 后端的订阅模块，需要从“按订阅创建任务”的思路，调整为“按影院创建轮询任务”的思路。

核心目标是：

1. 以影院为轮询中心

猫眼接口的入口是 `city -> cinema list -> cinemaDetail(cinemaId)`，`cinemaDetail` 会返回该影院的全部热映电影、排片和票价，因此轮询的最小单位应该是影院，不是电影，也不是用户。

2. 避免同影院重复请求猫眼接口

同一个影院下如果存在多个订阅规则、多个邮箱、多个电影条件，不应该创建多个定时任务重复执行；应当只保留一个影院轮询任务，采集一次结果后再分发给多个订阅规则判断。

3. 把“采集”“判断”“通知”“历史记录”拆开

让系统职责清晰，后续好扩展、好排查、好维护。

二、业务认知修正

当前订阅模块不要理解成“用户订阅电影”，而要理解成：

用户对某个影院的票价变化进行订阅，并通过邮箱接收通知。

也就是说：

- **影院**：数据采集中心
- **电影**：影院接口返回的结果数据，不是任务入口
- **订阅**：业务规则
- **邮箱**：通知接收方式
- **定时任务**：围绕影院建立的轮询任务
- **执行日志**：记录每次轮询结果
- **票价历史**：记录每次采集到的场次价格快照

三、核心设计原则

1. 影院是任务中心

一个影院对应一个轮询任务，任务只负责抓取这个影院的全部排片与票价。

2. 订阅只是规则，不是任务本身

订阅规则只描述：

- 订阅哪个影院
- 目标价格是多少
- 通知邮箱是谁
- 是否启用通知

订阅规则不直接负责请求猫眼接口。

3. 一次采集，多个订阅共享

同一个影院的数据只抓一次，然后把抓取结果交给该影院下的所有订阅规则去匹配，避免重复请求。

4. 历史记录与执行记录必须分离

- 执行日志：回答“这次任务跑没跑成功”
- 票价历史：回答“这次抓到了什么票价数据”

5. 修改订阅不影响历史

修改目标价格、邮箱、启停状态，只影响未来行为，不回改历史数据。

四、建议的领域模型

1. City

城市基础信息，用于定位影院列表。

2. Cinema

影院基础信息，是整个系统的中心对象。

3. Subscription

订阅规则，表示某个邮箱对某个影院的订阅配置。

4. CrawlTask

影院轮询任务，一个影院最多一个任务。

5. ExecuteLog

任务执行日志，记录每次轮询过程和结果。

6. PriceSnapshotBatch

一次轮询产生的快照批次。

7. PriceSnapshotItem

批次下的具体票价明细，按电影、场次、时间、价格进行落库。

8. NotifyLog

发送通知结果日志，用于排查邮件发送情况。

五、推荐的数据表思路

1. cinema

影院基础表。

建议字段：

- id
- city_id
- maoyan_cinema_id
- name
- address
- status
- created_at
- updated_at

用途：

- 保存影院基础信息
 - 作为轮询任务和订阅规则的挂载中心
-

2. subscription

订阅规则表。

建议字段：

- id
- cinema_id
- email
- target_price
- notify_enabled
- status
- baseline_min_price
- baseline_max_price
- created_at
- updated_at

用途：

- 一条记录表示一个邮箱对一个影院的订阅规则
- 保存创建订阅时该影院的基线票价范围
- 只修改目标价格、邮箱、启停状态，不直接影响历史数据

唯一约束建议

如果业务要求“一个影院只能绑定一个邮箱”，可加唯一约束：

- `cinema_id UNIQUE`

如果未来允许一个影院多个邮箱，则不要加这个唯一约束，而应改为：

- `UNIQUE(cinema_id, email)`

当前从你的描述看，更合理的是：

- 一个影院只维护一份轮询任务
- 一个影院下可以有多个订阅规则/邮箱
- 但同一个影院不应被重复创建多个采集任务

3. `crawl_task`

影院轮询任务表。

建议字段：

- `id`
- `cinema_id`
- `interval_minutes`
- `next_run_at`
- `last_run_at`
- `status`
- `created_at`
- `updated_at`

用途：

- 只负责影院级别的轮询调度
- 一个影院最多一条任务
- 任务创建后定时执行，抓取该影院的全部排片与价格

唯一约束建议

- `cinema_id UNIQUE`

这能确保同一个影院不会被创建多个轮询任务。

4. `execute_log`

执行日志表。

建议字段：

- `id`
- `crawl_task_id`
- `cinema_id`
- `started_at`
- `ended_at`
- `status` (success / fail / partial)
- `error_code`
- `error_msg`
- `fetches_count`
- `matched_count`
- `notified_count`
- `summary_json`
- `created_at`

用途：

- 记录每次任务执行是否成功
- 记录失败原因
- 记录本次抓到了多少条排片
- 记录有多少条命中阈值并触发通知

这个表是排障核心。

5. `price_snapshot_batch`

票价快照批次表。

建议字段：

- `id`
- `crawl_task_id`
- `cinema_id`
- `fetches_at`
- `source` (maoyan)
- `request_params_json`
- `raw_response_json` (可选，量大时可不存)
- `created_at`

用途：

- 表示“一次轮询抓取”的整体批次
 - 与具体明细分开，方便扩展和统计
-

6. price_snapshot_item

票价快照明细表。

建议字段：

- id
- batch_id
- cinema_id
- movie_id
- movie_name
- show_start_at
- show_end_at
- hall_name
- language
- version
- price
- original_price
- observed_at
- created_at

用途：

- 存储每次轮询抓取到的具体场次和票价
- 适合后续做票价趋势、历史回溯、最低价分析

设计重点

猫眼一次返回的是“影院多个电影、多个场次、多个日期”的信息，因此不要只存一个“最低价”字段就结束，应该按**场次粒度**存明细，后续才能做趋势分析和精准判断。

7. notify_log

通知日志表。

建议字段：

- id
- subscription_id
- execute_log_id
- email
- notify_status
- notify_content
- error_msg
- sent_at
- created_at

用途：

- 记录每次邮件通知是否发送成功

- 方便排查邮件发送失败、退信、重复通知等问题
-

六、推荐的业务流程

1. 新增订阅

流程建议：

1. 校验影院是否存在
2. 创建或复用 `crawl_task`
3. 创建 `subscription`
4. 立刻进行一次初始化抓取
5. 保存 `execute_log`
6. 保存 `price_snapshot_batch`
7. 保存 `price_snapshot_item`
8. 根据抓取结果判断是否命中目标价格
9. 命中则发送通知并记录 `notify_log`

2. 修改订阅

只允许修改：

- `target_price`
- `email`
- `notify_enabled`
- `status`

不修改：

- 影院轮询任务本身
- 票价历史
- 执行历史

3. 定时轮询

每 30 分钟执行一次：

1. 查出所有启用中的 `crawl_task`
 2. 按影院逐个请求猫眼 `cinemaDetail(cinemaId)`
 3. 解析返回的全部电影、排片、票价
 4. 写入 `execute_log`
 5. 写入 `price_snapshot_batch`
 6. 写入 `price_snapshot_item`
 7. 查出该影院下所有启用中的 `subscription`
 8. 对每条订阅规则判断是否达到目标价格
 9. 达到则发送通知
 10. 写入 `notify_log`
-

七、去重逻辑的关键点

当前你担心的问题本质上不是数据库，而是重复采集。

例如：

- 任务 A：影院A + 邮箱A + 电影1
- 任务 B：影院A + 邮箱A + 电影2

如果这两个任务都独立请求猫眼，就会重复抓取同一个影院数据两次。

正确做法是：

- 影院A 只建立一个 `crawl_task`
- 影院A 下的所有订阅规则共享这次采集结果
- 电影只是返回数据，不参与任务创建逻辑

换句话说：

轮询任务属于影院，不属于订阅。

八、订阅规则和任务的关系建议

建议把关系理解为：

- `crawl_task`：负责采集
- `subscription`：负责判断和通知

它们是两层职责，不要混在一起。

正确方式

- 一个影院 -> 一个采集任务
- 一个影院 -> 多个订阅规则（可选）
- 一个采集结果 -> 多个订阅规则共享判断

不推荐方式

- 每个订阅都创建一个任务
 - 每个邮箱都单独请求一次影院接口
 - 每个电影都独立拉一次影院数据
-

九、关于“一个影院只能提供给一个邮箱订阅”的理解

这个约束可以作为当前版本的业务规则，但建议从系统设计上把它理解为：

- 一个影院只有一个采集任务
- 一个影院的订阅规则可以按业务限制控制数量

- 现在可限制为一个邮箱
- 未来如果要扩展多个邮箱，只需要放开 `subscription` 的唯一约束即可

也就是说，限制的是订阅规则数量，不是轮询任务数量。

十、对现有 Go 后端模块的改造建议

1. 拆分现有订阅逻辑

将当前可能混在一起的逻辑拆成：

- 任务调度层
- 数据采集层
- 订阅匹配层
- 邮件通知层
- 日志持久化层

2. 修改“新增订阅”接口逻辑

新增订阅时不直接创建新的影院轮询任务，而是：

- 先检查该影院是否已有 `crawl_task`
- 有则复用
- 无则创建
- 然后创建订阅规则

3. 修改“删除订阅”接口逻辑

删除订阅时：

- 只删除或停用订阅规则
- 不要立刻删除影院任务
- 只有当该影院下所有订阅都不存在时，才考虑停用/删除对应任务

4. 修改“定时任务”逻辑

轮询任务的执行逻辑应该只按影院跑，不按订阅跑。

5. 增加日志链路

每次执行必须能追踪：

- 哪个影院执行了
 - 执行成功还是失败
 - 抓到了哪些数据
 - 哪些订阅命中
 - 哪些邮箱发送成功或失败
-

十一、推荐的最终结构一句话概括

影院是采集中心，订阅是规则，轮询任务属于影院，票价历史和执行日志独立存储，通知只在采集结果基础上进行分发。

十二、给 Agent 的执行目标摘要

请根据以上原则重构 Go 后端订阅模块，重点完成以下工作：

1. 调整数据模型：以 `cinema` 和 `crawl_task` 为中心
2. 拆分订阅规则与采集任务
3. 保证同一影院只会有一个轮询任务
4. 采集结果按批次和明细分开存储
5. 增加执行日志、票价快照、通知日志
6. 修改新增/修改/删除订阅逻辑
7. 修改定时任务调度逻辑，避免重复抓取同一影院
8. 保留历史数据，不允许订阅修改污染历史记录
9. 所有通知判断都基于同一次影院采集结果进行复用